f Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester V)

APPLIED BUSINESS M2

Practical – 11

Roll No.: T029	Name: Vinayak Padhy
Class:TY.IT	Batch:1
Date of Assignment:03-10-26	Date/Time of Submission:03-10-26

Practical 11: Optimization Modeling and Simulation Modeling

Part A: Optimization Modeling

Business Problem: A company produces two products, Product A and Product B. Thecompany wants to maximize its profit using the available machine and labour hours.

Given data:

Product Profit per Unit Machine Hours Labour Hours

A 40 2 1

B 30 1 2

Available Machine Hours = 100

Available Labour Hours = 80

Practical Questions:

1. Define the decision variables for Product A and Product B.

Let:

- x = Number of units of Product A
- y = Number of units of Product B

2. Define the objective of the problem.

The objective is to **maximize the total profit** from producing Product A and Product B.

3. Formulate the objective function to maximize the total profit.

Profit from Product A = 40x

Profit from Product B = 30y

Therefore:

Maximize:

Z = 40x + 30y

4. Formulate the machine-hour constraint.

Product A requires 2 machine hours and Product B requires 1 machine hour.

Available machine hours = 100.

Therefore: **2x + y ≤ 100**

5. Formulate the labour-hour constraint.

Product A requires 1 labour hour and Product B requires 2 labour hours.

Available labour hours = 80.

Therefore:**x + 2y ≤ 80**

6. Add non-negative constraints for the decision variables.

Number of products cannot be negative:

x ≥ 0

y ≥ 0

7. Solve the linear programming problem using Python.

INPUT:-

```
from scipy.optimize import linprog
c = [-40, -30] A = [ [2, 1],   # Machine hours [1, 2]    # Labour hours]
b = [100, 80]
bounds = [ (0, None),
   (0, None)   result = linprog(   c,    A_ub=A,    b_ub=b,
   bounds=bounds,  method="highs")print(result)
```

OUTPUT:-

```
         message: Optimization terminated successfully. (HiGHS Status 7: Optimal)
         success: True
          status: 0
             fun: -2200.0
               x: [ 4.000e+01  2.000e+01]
             nit: 2
           lower:  residual: [ 4.000e+01  2.000e+01]
                 marginals: [ 0.000e+00  0.000e+00]
           upper:  residual: [       inf        inf]
                 marginals: [ 0.000e+00  0.000e+00]
           eqlin:  residual: []
                 marginals: []
         ineqlin:  residual: [ 0.000e+00  0.000e+00]
                 marginals: [-1.667e+01 -6.667e+00]
  mip_node_count: 0
  mip_dual_bound: 0.0
         mip_gap: 0.0
```

8. Find the optimal number of units of Product A and Product B.

INPUT:-

```
x = result.x[0]
y = result.x[1]
print("Optimal units of Product A:", x)
print("Optimal units of Product B:", y)
```

OUTPUT:-

```
Optimal units of Product A: 40.0
Optimal units of Product B: 20.0
```

9. Calculate the maximum profit.

INPUT:-

```
maximum_profit = 40 * x + 30 * y
print("Maximum Profit:", maximum_profit)
```

OUTPUT:-

```
Maximum Profit: 2200.0
```

10. Check whether the available machine and labour hours are satisfied.

INPUT:-

```
machine_used = 2 * x + y
labour_used = x + 2 * y
print("Machine hours used:", machine_used)
print("Available machine hours:", 100)
print("Labour hours used:", labour_used)
print("Available labour hours:", 80)
```

OUTPUT:-

```
Machine hours used: 100.0
Available machine hours: 100
Labour hours used: 80.0
Available labour hours: 80
```

11. Display the optimal solution in a suitable format.

INPUT:-

```
print("----- Optimal Solution -----")
print("Product A:", int(x), "units")
print("Product B:", int(y), "units")
print("Maximum Profit:", int(maximum_profit))
print("Machine Hours Used:", int(machine_used))
print("Labour Hours Used:", int(labour_used))
```

OUTPUT:-

```
----- Optimal Solution -----
Product A: 40 units
Product B: 20 units
Maximum Profit: 2200
Machine Hours Used: 100
Labour Hours Used: 80
```

12. Interpret the optimal solution in a business context.

The company should produce 40 units of Product A and 20 units of Product B. This production plan uses all 100 available machine hours and 80 available labour hours and gives a maximum profit of 2200.

13. Write a conclusion based on the optimization result.

The linear programming model determines the optimal production combination of Product A and

Product B. The optimal solution is 40 units of Product A and 20 units of Product B, resulting in a maximum profit of 2200 while satisfying all machine and labour constraints. Part B: Simulation Modeling

Business Problem: A supermarket wants to decide the appropriate inventory level for a product when daily demand is uncertain.

Practical Questions:

14. Generate random daily demand values for a product for 30 days.

INPUT:-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(42)
demand = np.random.randint(20, 61, size=30)
print("Random Daily Demand for 30 Days:")
print(demand)
```

OUTPUT:-

```
Random Daily Demand for 30 Days:
[58 48 34 27 40 58 38 42 30 30 43 55 59 43 22 41 21 43 49 57 21 40 52 31
 41 44 46 47 35 34]
```

15. Display the generated demand values.

INPUT:-

```
days = np.arange(1, 31)
demand_df = pd.DataFrame({
   "Day": days,
   "Demand": demand})
print(demand_df)
```

OUTPUT:-

	Day	Demand
0	1	58
1	2	48
2	3	34
3	4	27
4	5	40
5	6	58
6	7	38
7	8	42
8	9	30
9	10	30
10	11	43
11	12	55
12	13	59
13	14	43

16. Select an inventory level of 30 units.

INPUT:-

```
inventory = 30
print("Inventory Level:", inventory)
```

OUTPUT:-

```
Inventory Level: 30
```

17. Compare the daily demand with the available inventory.

18. Calculate the number of units sold each day.

19. Calculate the number of units left after each day.

20. Calculate the shortage when demand is greater than available inventory.

INPUT:-

```
inventory = 30
units_sold = []
units_left = []
shortage = []
for daily_demand in demand:
   sold = min(daily_demand, inventory)
    left = inventory - sold
   short = max(daily_demand - inventory, 0)
   units_sold.append(sold)
   units_left.append(left)
   shortage.append(short)
simulation_30 = pd.DataFrame({
   "Day": days,
   "Demand": demand,
   "Inventory": inventory,
   "Units Sold": units_sold,
   "Units Left": units_left,
   "Shortage": shortage})
print(simulation_30)
```

OUTPUT:-

	Day	Demand	Inventory	Units Sold	Units Left	Shortage
0	1	58	30	30	0	28
1	2	48	30	30	0	18
2	3	34	30	30	0	4
3	4	27	30	27	3	0
4	5	40	30	30	0	10
5	6	58	30	30	0	28
6	7	38	30	30	0	8
7	8	42	30	30	0	12
8	9	30	30	30	0	0
9	10	30	30	30	0	0
10	11	43	30	30	0	13
11	12	55	30	30	0	25
12	13	59	30	30	0	29
13	14	43	30	30	0	13
14	15	22	30	22	8	0
15	16	41	30	30	0	11
16	17	21	30	21	9	0
17	18	43	30	30	0	13
18	19	49	30	30	0	19
19	20	57	30	30	0	27
20	21	21	30	21	9	0
21	22	40	30	30	0	10
22	23	52	30	30	0	22
23	24	31	30	30	0	1
24	25	41	30	30	0	11

21. Calculate the total units sold and total shortage for 30 days.

INPUT:-

```
total_sales_30 = simulation_30["Units Sold"].sum()
total_shortage_30 = simulation_30["Shortage"].sum()
print("Inventory Level:", 30)
print("Total Units Sold:", total_sales_30)
print("Total Shortage:", total_shortage_30)
```

OUTPUT:-

```
Inventory Level: 30
Total Units Sold: 871
Total Shortage: 358
```

22. Repeat the simulation using inventory levels of 30, 40, and 50 units.

INPUT:-

```
inventory_levels = [30, 40, 50]
results = []
for inventory in inventory_levels:
  total_sold = 0
   total_shortage = 0
   for daily_demand in demand:
      sold = min(daily_demand, inventory)
      short = max(daily_demand - inventory, 0)
      total_sold += sold
      total_shortage += short
   results.append({
      "Inventory Level": inventory,
      "Total Units Sold": total_sold,
      "Total Shortage": total_shortage    })
```

print(results)

OUTPUT:-

```
[{'Inventory Level': 30, 'Total Units Sold': np.int64(871), 'Total Shortage': np.int
```

23. Store the results of all three scenarios in a DataFrame.

INPUT:-

```
results_df = pd.DataFrame(results)
print(results_df)
```

OUTPUT:-

```
   Inventory Level  Total Units Sold  Total Shortage
0               30               871             358
1               40              1083             146
2               50              1190              39
```

24. Compare the total sales and total shortage for each inventory level.

INPUT:-

```
print("Comparison of Inventory Levels:")
print(results_df)
```

OUTPUT:-

```
Comparison of Inventory Levels:
   Inventory Level  Total Units Sold  Total Shortage
0               30               871             358
1               40              1083             146
2               50              1190              39
```

Observation:

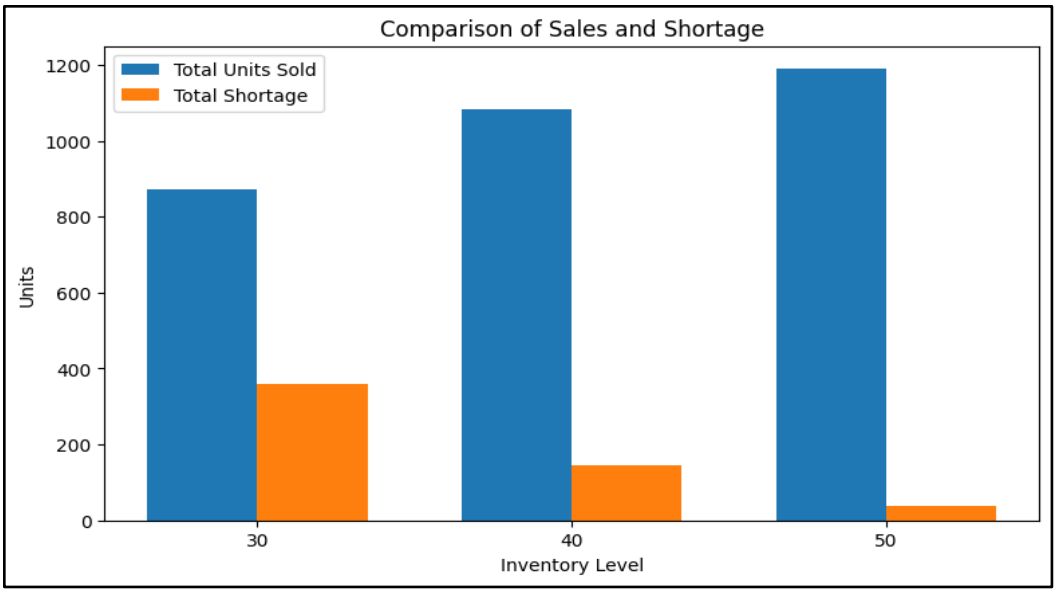
- When inventory level increases, the number of units sold generally increases.
- When inventory level increases, the total shortage decreases.

25. Create a chart to compare the results of the three scenarios.

INPUT:-

```
x = np.arange(len(results_df))
width = 0.35
plt.figure(figsize=(9, 5))
plt.bar(   x - width/2,
   results_df["Total Units Sold"],
   width,
   label="Total Units Sold")
plt.bar( x + width/2,
   results_df["Total Shortage"],
   width,
   label="Total Shortage")
plt.xlabel("Inventory Level")
plt.ylabel("Units")
plt.title("Comparison of Sales and Shortage")
plt.xticks(x, results_df["Inventory Level"])
plt.legend()plt.show()
```

OUTPUT:-



26. Identify how changing the inventory level affects sales and shortage.

Observation:

Increasing the inventory level from 30 to 40 and 50 units increases the number of units that can be sold when demand is higher than the available inventory. At the same time, the shortage decreases as more inventory is available.

27. Write a conclusion about the inventory level based on the simulation results.

Conclusion: The simulation shows that inventory level has a direct effect on sales and shortage. A lower inventory level can result in more shortages when daily demand is high. Increasing the inventory level reduces shortages and allows more units to be sold. Therefore, the supermarket can compare different inventory levels and select an appropriate level according to its demand requirements.